



Platformer 2D

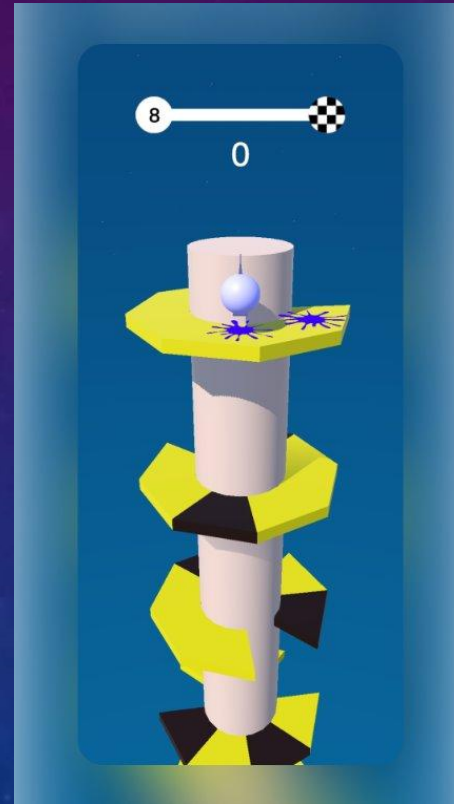
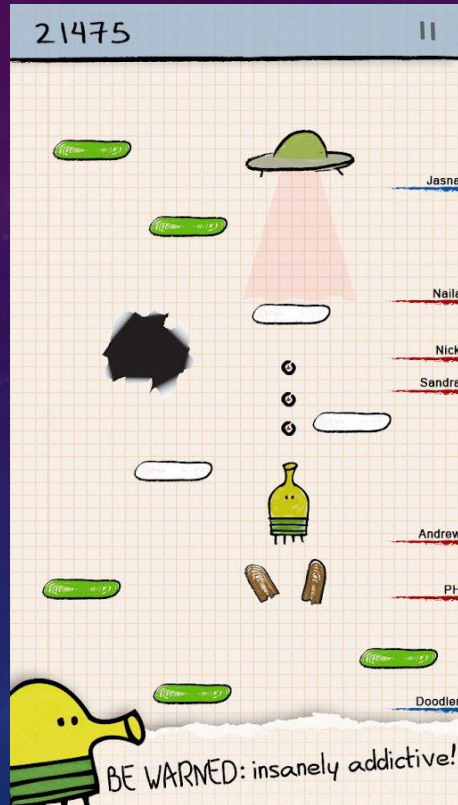
DEVERNAY Timothée
DUCRAY Vincent
LASADE Adrien
LELEGARD Nathan

Soutenance PRO3600

Plan

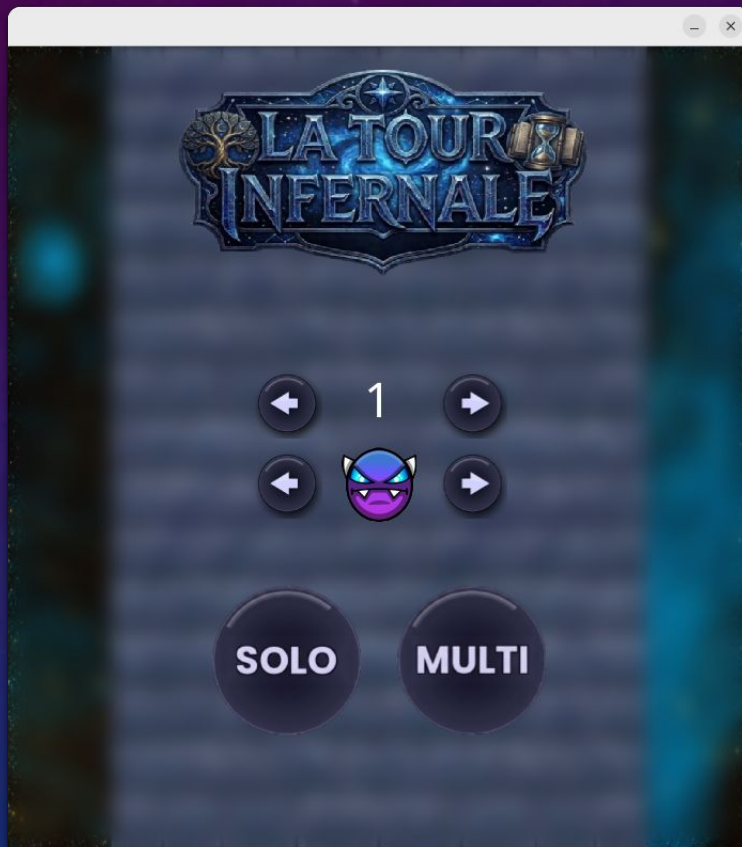
1. Introduction
2. Démonstration
3. Outils de gestion de projet
4. Répartition des tâches
5. Les technologies
6. Conception
7. Les tests

1/ L'introduction - l'idée



Timothee

2/ Démonstration



Timothée

3/ Outils de gestion de projet - Chronologie du projet

Gantt timeline



3/ Outils de gestion de projet - Les outils de collaboration



3/ Outils de gestion de projet Répartition du travail

<> Code Issues 16 Pull requests Agents Actions Projects Security and quality Insights Settings

is:issue state:open 🔍 Labels Milestones New issue

☐ Open 16 Closed 20 Author Labels Projects Milestones Assignees Newest

- ☐  Effet 3D des plateformes à gauche du poteau
#116 · nlelegard opened last month 
- ☐  Nettoyer le code
#113 · T4683842 opened last month 
- ☐  Private attribute (utiliser getter Asset)
#78 · Vince51350 opened on Apr 1 
- ☐  Répertoire code/src
#77 · Vince51350 opened on Apr 1 
- ☐  Différentes scènes
#75 · Vince51350 opened on Apr 1  1
- ☐  Changement de fond selon altitude
#58 · Vince51350 opened on Mar 3 

4/ Répartition des tâches

Adrien

Physique

Génération

Nathan

Overlays menu et
GameOver

Sons

Timothée

Ressources

Plateformes

Vincent

Score

Mort

Architecture

Point de départ



CSC 3101 – Algorithmique et langage de programmation

Portail informatique

► Liens utiles ► Supports ► Les algorithmes

CI9 : Alien vs ananas

*Objectifs : L'objectif de cet exercice est de vous faire manipuler des classes anonymes et de vous introduire la bibliothèque **JavaFX** qui permet de créer facilement des applications graphiques.*

Durée des exercices obligatoires : 2h30mn en présentiel

- Exercice 1 : *obligatoire* (facile, ~10mn)
- Exercice 2 : *obligatoire* (facile, ~45mn)
- Exercice 3 : *obligatoire* (facile, ~30mn)
- Exercice 4 : *obligatoire* (moyen, ~30mn)
- Exercice 5 : *obligatoire* (moyen, ~45mn)

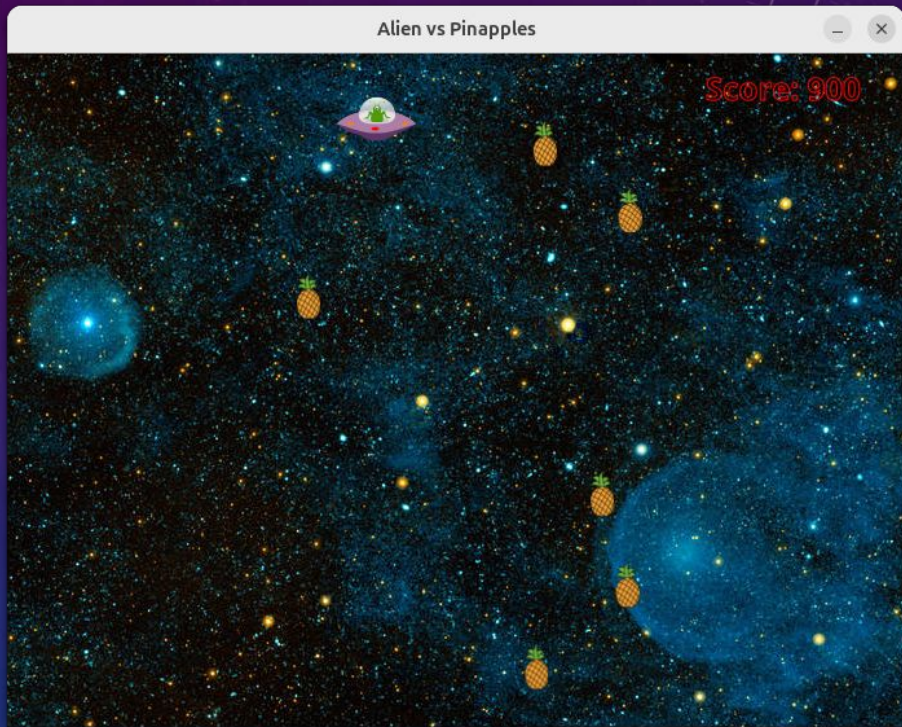
Dans cet exercice, nous concevons un jeu vidéo simple nommé Alien vs Pinapples : une armée d'ananas mutants attaque votre galaxie et vous jouez un alien en charge de sauver l'univers, ytpa !

Exercice 1 : Préparation (~10mn – facile – obligatoire)

Avant de pouvoir programmer votre jeu, vous devez préparer votre environnement de travail afin d'utiliser la bibliothèque graphique **JavaFX**. Cette activité devrait vous prendre environ 10 minutes si tout se passe bien.

Si vous utilisez IntelliJ, créez un nouveau projet de la manière suivante. Dans la fenêtre de création de projet, sur la gauche, sélectionnez JavaFX. Nommez le projet **alien**. Créez le projet. Il vous sera demandé quelles bibliothèques installer. Sélectionnez tout. Dans le projet créé, supprimez le dossier **com** (qui contient **exemple/alien** et trois classes). Ouvrez le **fichier `src/main/java/module-info.java`** et remplacez **`com.example`** par **`tsp`**. Créez la classe **Alien** dans le package **`tsp.alien`** et collez-y le code suivant :

5/ Les Technologies



5/ Les Technologies



stockage des couples clés-valeurs

```
vincent@vincent-HP-Pavilion-Laptop-15-eg2xxx:~/tsp_plateformer$ cat scores.properties
#Best scores by seed
#Wed Jun 10 21:32:12 CEST 2026
0=27
1=22
2=21
```



ouverture, lecture, écriture de fichier

5/ Les Technologies : imports supplémentaires



Javax : extension de Java

Javax.sound.sampled



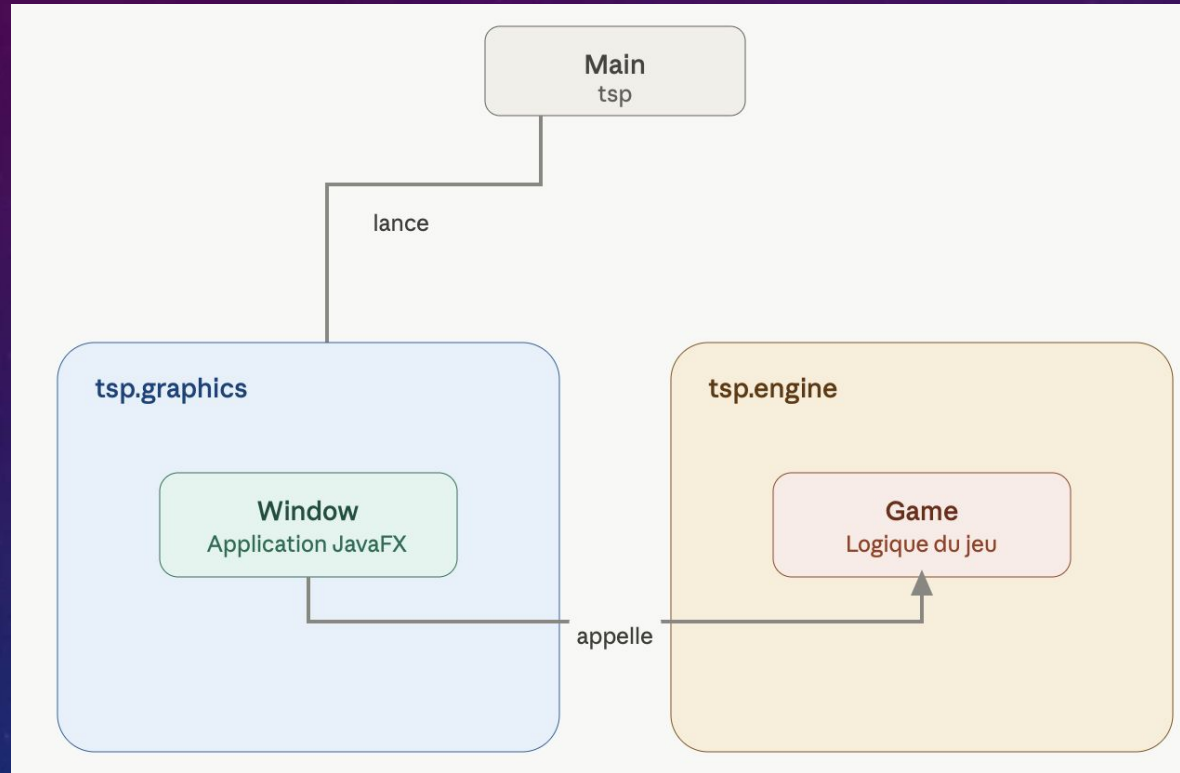
—> Audiosystem

5/ Les technologies : Utilisation de l'intelligence artificielle



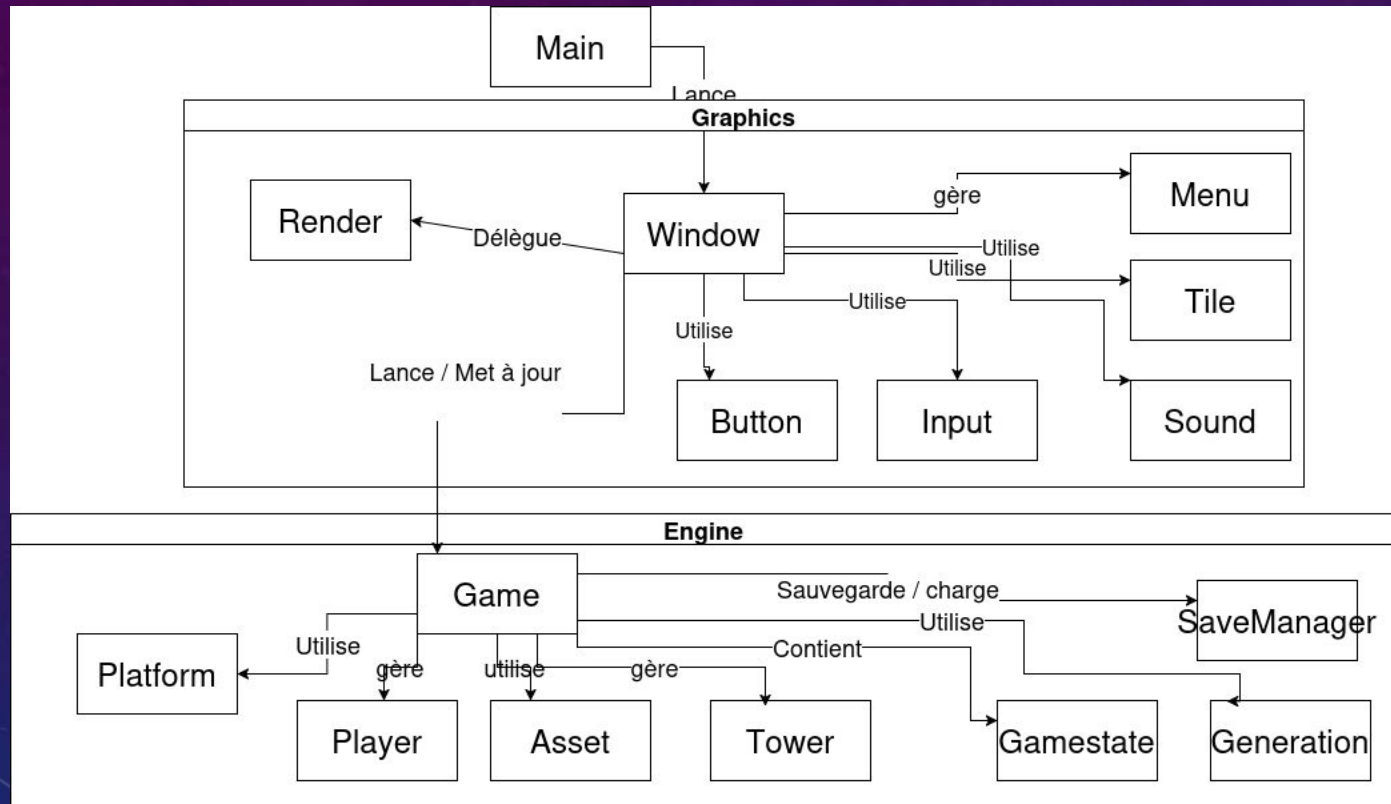
6/ Conception

Diagramme architecture

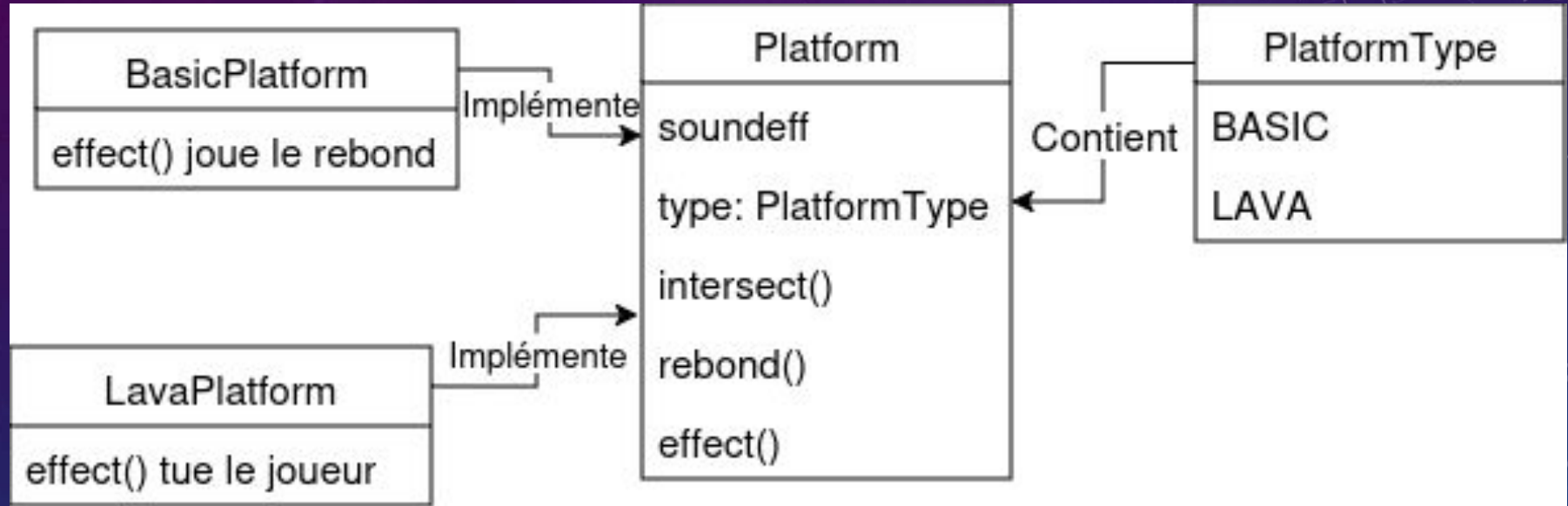


6/ Conception

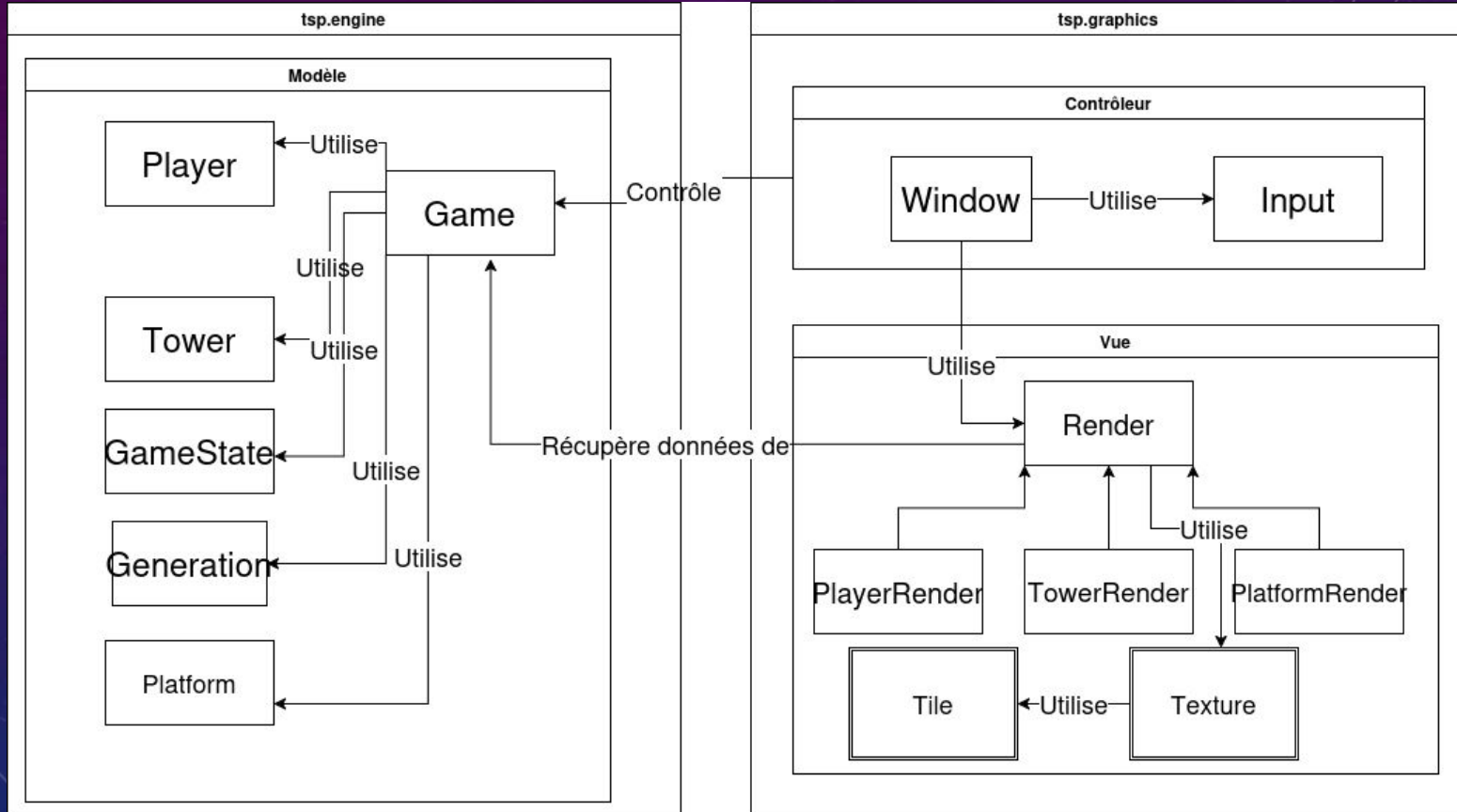
Diagramme architecture



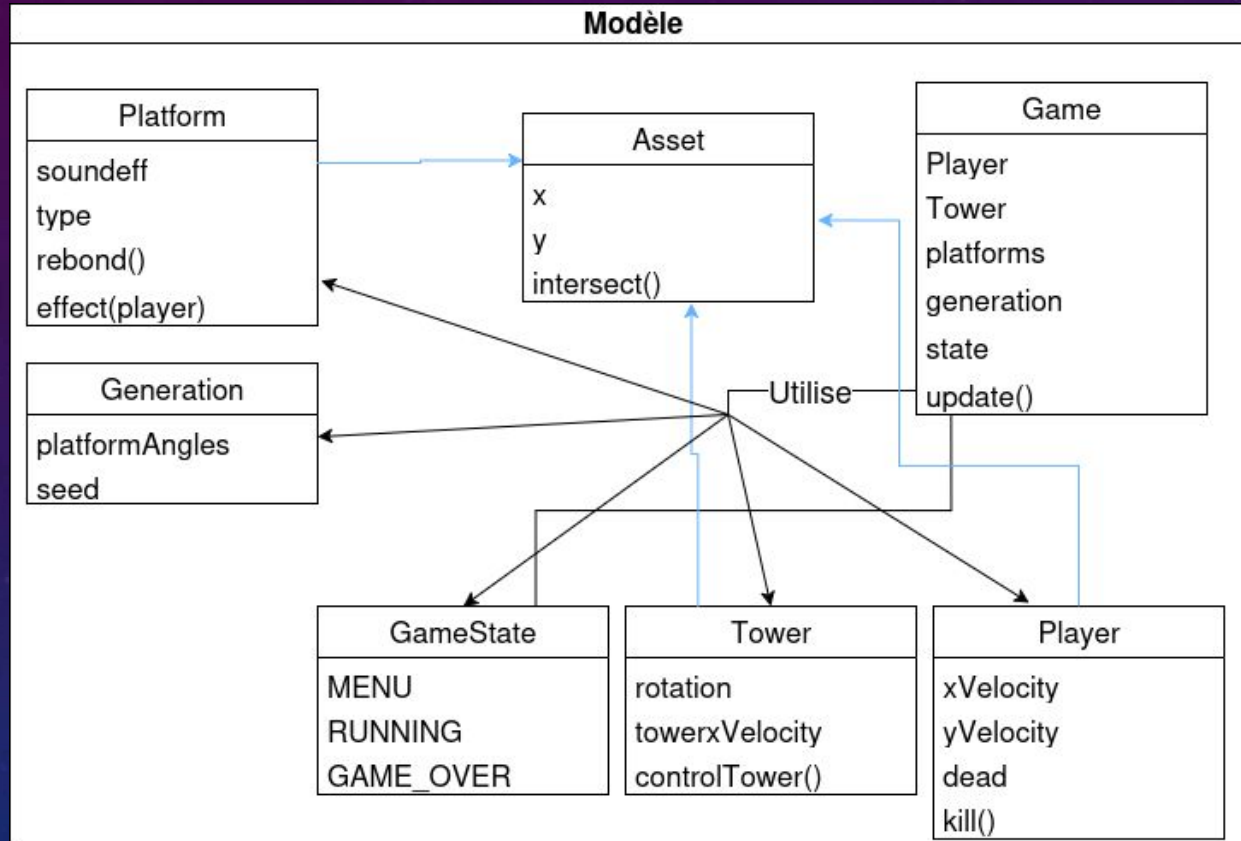
6/ Conception - Héritage



6/ Conception - Modèle vue contrôleur



6/ Conception - Diagramme détaillé du modèle

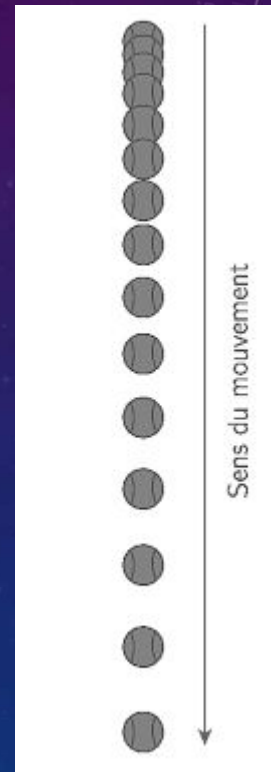


6/ Conception - La physique

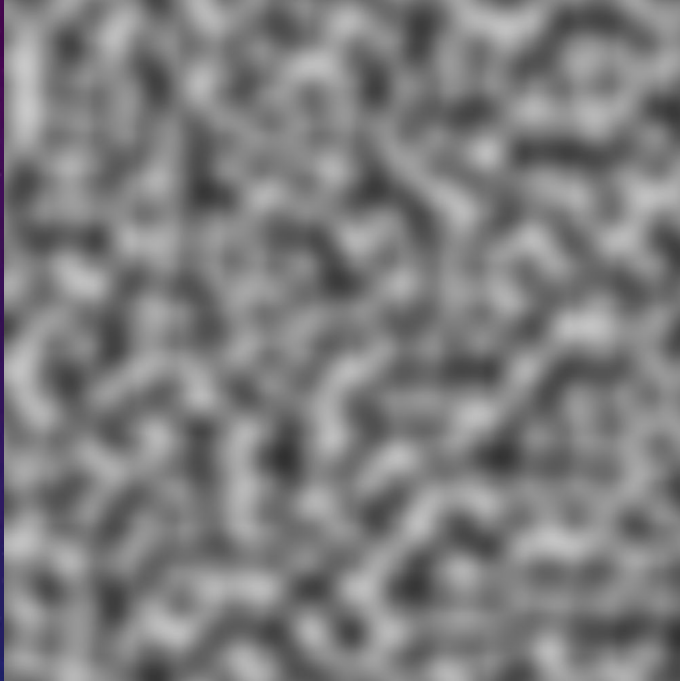
```
/**  
 * Vitesse gagnée par le Player  
 */  
private double xVelocity=0;  
private double yVelocity=0;
```

Le cœur de tout moteur physique :

$$x(t + \Delta t) = x(t) + v \cdot \Delta t$$



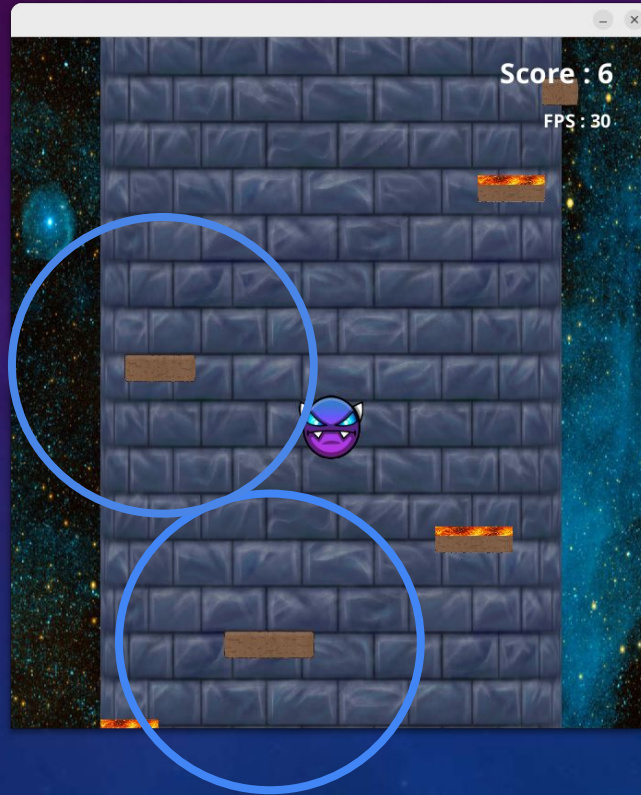
6/ Conception - La génération des niveaux



Perlin noise
source: wikimedia

```
// create random object  
Random randomNumbers = new Random();
```

6/ Conception - La génération des niveaux



Adrien

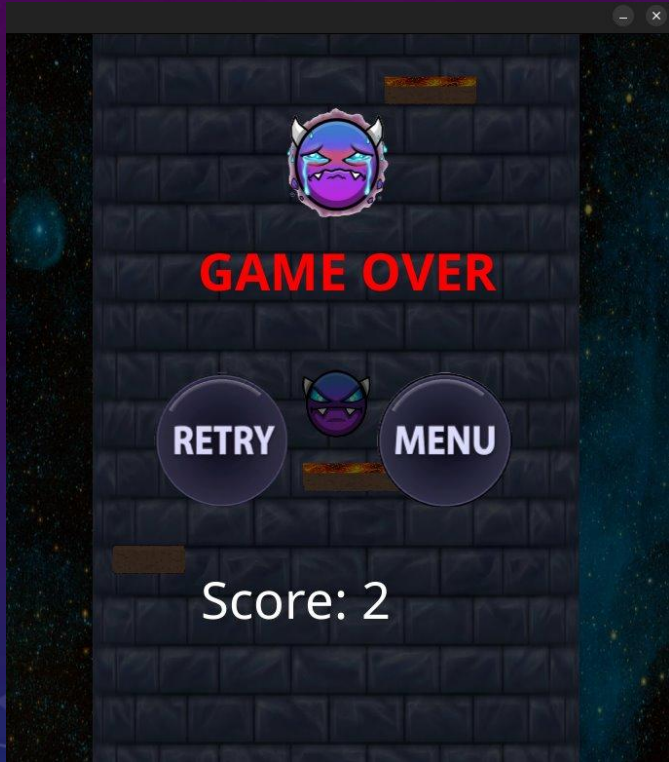
6/ Conception - Effet 3d des plateformes



6/ Conception - Effet 3d des plateformes

```
for(Platform platform1 : platforms) {  
    int relRotation = ((int)rotation - generator.getPlatformAngles()[i] + 360) % 360;  
    double cosRel = Math.cos(relRotation * 2 * 3.14159 / 360);  
    double sinRel = Math.sin(relRotation * 2 * 3.14159 / 360);  
  
    if(relRotation > 0 && relRotation < 180) {  
        double platformWidth = Math.abs(sinRel) * 100;  
        double newX = towercenterx + cosRel * tower.getWidth() - platformWidth * (1 - cosRel) / 2;  
  
        platform1.setPosition(newX, platform1.getY());  
        this.renderPlatform(platform1, platformWidth, platform1.getHeight());  
    }  
    i++;  
}
```

6/ Conception - Les Menus



À chaque frame :

Affichage selon le mode de jeu

Affichage de Menu + GameOver

ssi : Franchissement du bouton

⇒ Utilisation d'un booléen

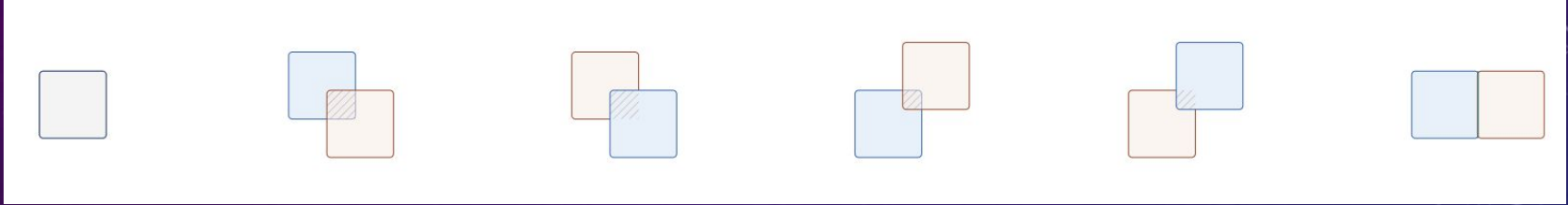
7/ Les Tests - Test d'intersection

JUnit 5

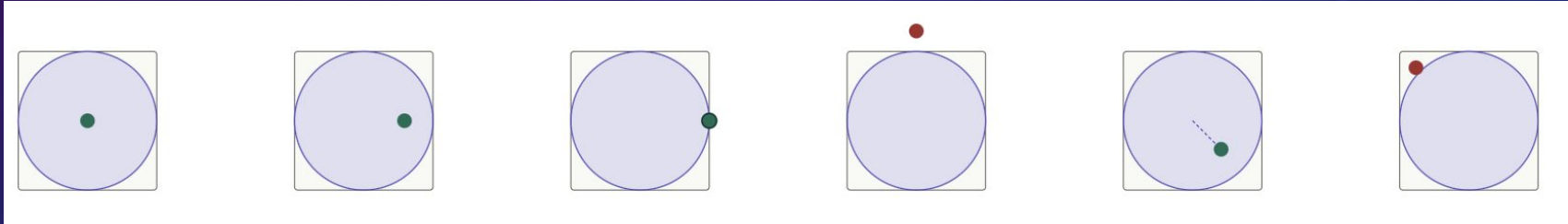


7/ Les Tests

intersect()



isHoveredCirc()



Ce qu'on aurait aimé implémenter

- Comète
- Plateformes
 - Tombante après 1 rebond
 - En ressort
- Une fin (un drapeau)
- Un Menu pause (depuis RUNNING)





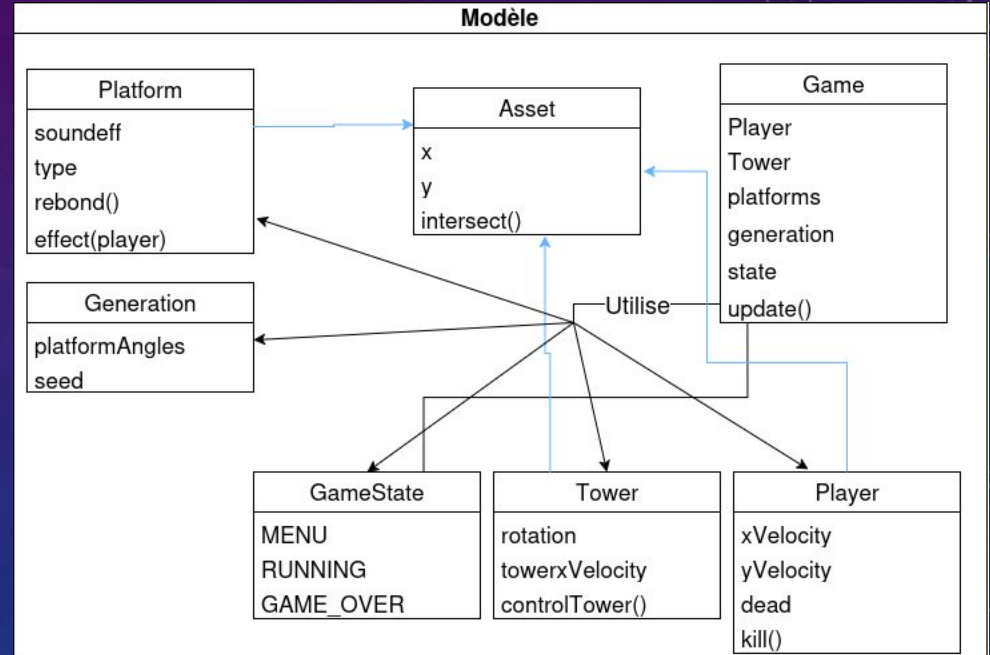
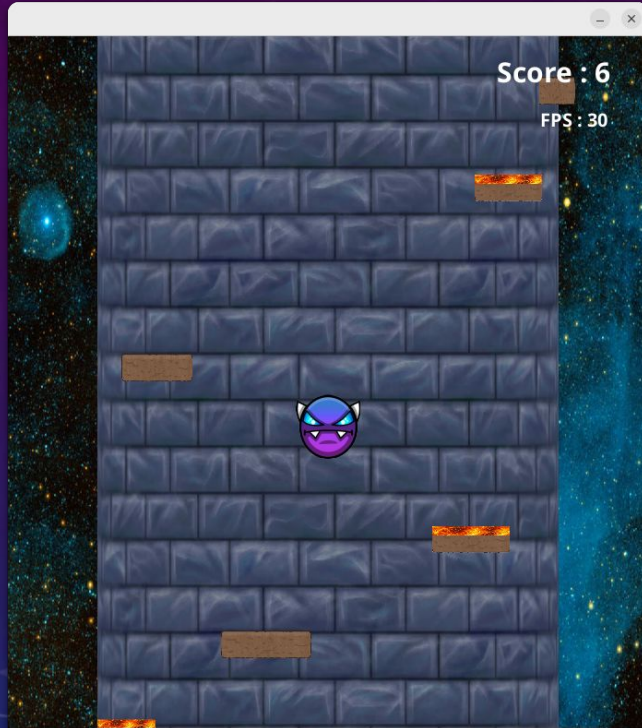
FIN

RETRY



MENU

Questions/Réponses



Annexes

```
@DisplayName("Test 1: Superposition exacte (même position)")  
void exactOverlap() {  
    playerUnderTest1.setPosition(100, 100);  
    playerUnderTest2.setPosition(100, 100);  
    Assertions.assertTrue(playerUnderTest1.intersects(playerUnderTest2));  
}
```

Annexes

```
@Test
@DisplayName("Test 2: Chevauchement partiel - other en bas à droite")
void overlapBottomRight() {
    playerUnderTest1.setPosition(100, 100);
    playerUnderTest2.setPosition(140, 140); //  $\Delta=(40,40) < 70$ 
    Assertions.assertTrue(playerUnderTest1.intersects(playerUnderTest2));
}
```


Annexes

```
@DisplayName("isHovered - Test 1: point au centre => true")  
void hoveredCenter() {  
    Assertions.assertTrue(button.isHovered(200, 200));  
}
```


Annexes

```
@DisplayName("isHovered - Test 2: coin haut-gauche exact => true")  
void hoveredTopLeftCorner() {  
    Assertions.assertTrue(button.isHovered(100, 100));  
}
```

Annexes

Dans Window.java :

Méthode :

handleClick

```
public void handleClick(double sourisx, double sourisy) {  
    if (game.getState() == GameState.MENU) {  
        if (menu.isPlayingClicked(sourisx, sourisy)) {  
            final double startingYpos = this.getHeight()-PlatformSpacing-150;  
            game.getPlayer().setPosition(game.getPlayer().getX(),startingYpos); //Replace le Player en position de départ  
            game.generateSeed(this);  
            platformRender.setPlatforms(game.getPlatforms());  
            game.setState(GameState.RUNNING);  
        }  
        if (menu.isLeftArrowClicked(sourisx, sourisy)) {  
            playerRender.previousSkin();  
            menuNeedsRedraw = true;  
        }  
        if (menu.isRightArrowClicked(sourisx, sourisy)) {  
            playerRender.nextSkin();  
            menuNeedsRedraw = true;  
        }  
        if (menu.isSeedLeftArrowClicked(sourisx, sourisy)) {  
            game.previousSeed();  
            menuNeedsRedraw = true;  
        }  
        if (menu.isSeedRightArrowClicked(sourisx, sourisy)) {  
            game.nextSeed();  
            menuNeedsRedraw = true;  
        }  
    }  
    else if (game.getState() == GameState.GAME_OVER) {  
        if (gameOver.isRetryClicked(sourisx,sourisy)) {  
            int savedSkin = playerRender.getCurrentSkinIndex(); // Sauvegarde l'index  
            game.reset(this);  
            startRenderers();  
            playerRender.setSkinIndex(savedSkin); // Réapplique le skin après StartRenderers  
            ycamera = 0;  
            soundDeath.stopMusic(); // Arrête la musique du GameOver  
        }  
        if (gameOver.isMenuClicked(sourisx, sourisy)) {  
            game.getPlayer().revive();  
            startRenderers();  
            menuBackground = null;  
            menuNeedsRedraw = true; // Permet de redessiner le Menu  
            ycamera = 0; // Remet la caméra en bas de la tour  
            soundDeath.stopMusic(); // On coupe la musique  
            game.setState(GameState.MENU); // reset met RUNNING, on repasse en MENU  
        }  
    }  
}
```

Annexes

Dans Window.java :

Méthode :

setMenu()

```
private void setMenu() {
    // On n'affiche PAS le menu à CHAQUE frame
    if (!menuNeedsRedraw && !menu.doesMenuButtonsNeedsRedraw(sourisx, sourisy)) {
        return; // rien à redessiner
    }
    menuNeedsRedraw = false;
    // On ne calcule le snapshot flou qu'une seule fois
    if (menuBackground == null) {
        // On vide le graphical Context
        this.getGC().clearRect(0, 0, this.getWidth(), this.getHeight());
        // On dessine le fond et la tour
        this.getGC().drawImage(bg.getImage(), 0, 0, this.getWidth(), this.getHeight());
        towerRender.render();

        SnapshotParameters params = new SnapshotParameters();
        params.setFill(Color.TRANSPARENT);

        WritableImage snapshotTower = gameGroup.snapshot(params, null);
        WritableImage snapshotCanvas = this.getCanvas().snapshot(params, null);

        // On fusionne les deux snapshots floutés dans menuBackground
        menuBackground = new WritableImage(this.getWidth(), this.getHeight());
        Canvas tempCanvas = new Canvas(this.getWidth(), this.getHeight());
        GraphicsContext tempGc = tempCanvas.getGraphicsContext2D();
        tempGc.setEffect(menuBlur);
        tempGc.drawImage(snapshotCanvas, 0, 0);
        tempGc.drawImage(snapshotTower, 0, 0);
        tempGc.applyEffect(menuBlur);
        SnapshotParameters p2 = new SnapshotParameters();
        p2.setFill(Color.TRANSPARENT);
        tempCanvas.snapshot(p2, menuBackground); // On sauvegarde le résultat flouté
    }

    // Chaque frame : on redessine juste le fond mémorisé, pas de snapshot
    this.getGC().clearRect(0, 0, this.getWidth(), this.getHeight());
    this.getGC().drawImage(menuBackground, 0, 0);
    // On place le joueur au milieu de la fenetre
    game.getPlayer().setPosition(game.getPlayer().getX(), windowHeight / 2);

    menuCanvas.setVisible(true);

    playerRender.render(); // on affiche le joueur

    int seed = game.getGenerator().getSeed();
    int bestScore = saveManager.getBestScore(seed);
    menu.render(menuCanvas, seed, bestScore, sourisx, sourisy); // Affichage du Menu : Boutons + Titre
}
```

Annexes

Dans Player.java :

Méthode :

calculatePosition

```
public void calculatePosition(int windowsWidth,
    int windowsHeight, Platform[] platforms) {

    // Calcule la position du joueur à la prochaine frame.
    yVelocity+=gravity;
    xVelocity = 0.7 * xVelocity; // doit être identique au coefficient dans controlTower de tower.java
    yVelocity = 0.97 * yVelocity;

    // Bordure de l'écran et gestion des collisions
    if (x < 0) {
        xVelocity = 0;
        x = 0;
    }else if (x+width > windowsWidth){
        xVelocity = 0;
        x = windowsWidth-width;
    }

    if (y+height > windowsHeight){
        if (yVelocity>FALL_THRESHOLD) { // dégâts de chute (sol)
            this.kill();
        } else {
            y=windowsHeight-height;
            yVelocity=-playeryVelocity;
            ground=true;
        }
    }

    // Collision avec les plateformes
    for(Platform platform : platforms) {
        if(platform.intersects(this)) {
            if (yVelocity>FALL_THRESHOLD) { // dégâts de chute (plateforme)
                this.kill();
            } else if (yVelocity>0){
                yVelocity=-playeryVelocity;
                ground=true;
                platform.effect(this);
            }
        }
    }

    this.x = this.x + xVelocity;
    this.y = this.y + yVelocity;
}
```